PROMPT PARA CLAUDE CODE

Sistema de Logística de Entrega de Última Milla — Implementación Backend

---

CONTEXTO DEL SISTEMA

Eres un experto en arquitectura de software Python. Debes implementar el backend del **Sistema de Logística de Entrega de Última Milla en Tiempo Real**, un sistema cloud-native que gestiona la entrega de paquetes desde bodega hasta el cliente final.

El sistema resuelve el problema de incertidumbre en la ventana de entrega (sin ETA preciso), visitas fallidas por vivienda vacía y sobrecostos por reintentos. Transforma la operación de reactiva a proactiva mediante tracking GPS en tiempo real, cálculo dinámico de ETA y notificaciones push al cliente.

Actores del sistema:

- **Cliente Final:** Consulta estado, ETA y mapa en tiempo real de su paquete.
- **Repartidor Interno:** Recibe hoja de ruta digital, transmite GPS, registra entregas y novedades.
- **Gestor Logístico:** Monitorea operación en tiempo real, gestiona ANS/KPIs e incidencias.

OBJETIVO DE IMPLEMENTACIÓN

Implementar la lógica de negocio básica para los siguientes dominios funcionales:

1. **Registro de Paquetes** — administrado mediante colas SQS/SNS de AWS (patrón productor/consumidor).
2. **Seguimiento del Paquete** — consulta del estado actual y ubicación GPS del paquete.
3. **Motor de ETA** — cálculo y exposición del tiempo estimado de entrega (ETA dinámico).
4. **Servicio de Notificaciones** — patrón publicador/suscriptor con SNS de AWS.

Alcance de esta fase: Lógica básica que lea la base de datos y devuelva resultados. No se requieren implementaciones complejas ni cálculos avanzados.

STACK TECNOLÓGICO

Componente	Tecnología
Lenguaje	Python 3.12+
Framework web	Flask (última versión estable)
ORM	SQLAlchemy 2.x con PyMySQL
Base de datos	MySQL en AWS RDS
Mensajería	AWS SQS + SNS (boto3)
Contenedores	Docker + Docker Compose
Orquestación	Compatible con EKS (Kubernetes) y EC2 (agnóstico)
Configuración	Variables de entorno (.env) con python-dotenv

ARQUITECTURA: DOMAIN DRIVEN DESIGN (DDD)

Estructura de dominios

El proyecto se organiza en **4 dominios** independientes, cada uno con responsabilidad única:

```
ultima_milla/
│
├── packages/              # Dominio: Registro de Paquetes
│   ├── presentation/      # Endpoints Flask, schemas de request/response
│   ├── services/          # Lógica de negocio del dominio
│   ├── infrastructure/    # Acceso a datos (SQLAlchemy), clientes AWS
│   ├── domain/            # Modelos, entidades, value objects
│   ├── utils/             # Utilidades específicas del dominio
│   └── container.py       # Inyección de dependencias del dominio
│
├── tracking/              # Dominio: Seguimiento del Paquete
│   ├── presentation/
│   ├── services/
│   ├── infrastructure/
│   ├── domain/
│   ├── utils/
│   └── container.py
│
```

```
├── eta/                # Dominio: Motor de ETA
│   ├── presentation/
│   ├── services/
│   ├── infrastructure/
│   ├── domain/
│   ├── utils/
│   └── container.py
│
├── notifications/      # Dominio: Notificaciones
│   ├── presentation/
│   ├── services/
│   ├── infrastructure/
│   ├── domain/
│   ├── utils/
│   └── container.py
│
├── shared/             # Módulo transversal compartido
│   ├── database.py       # Configuración SQLAlchemy (engine, session)
│   ├── config.py         # Carga de variables de entorno
│   ├── exceptions.py     # Excepciones base del sistema
│   ├── http_responses.py # Helpers para respuestas HTTP estandarizadas
│   └── base_models.py    # Modelos base SQLAlchemy
│
├── app.py              # Factory de Flask, registro de blueprints
├── wsgi.py             # Entrypoint para producción (gunicorn)
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .env.example
└── README.md
```

Convención de capas por dominio

Cada capa dentro de un dominio tiene una responsabilidad única y se comunica con las capas adyacentes **exclusivamente a través de interfaces (clases abstractas ABC)**:

Capa	Responsabilidad	Comunica con
`presentation/`	Endpoints Flask, validación de entrada, formato de respuesta HTTP	Llama a `services/` via interfaz
`services/`	Lógica de negocio, orquestación, reglas del dominio	Llama a `infrastructure/` via interfaz
`infrastructure/`	Repositorios SQLAlchemy, clientes boto3 (SNS/SQS), acceso a datos	Implementa interfaces definidas en `domain/`

`domain/`	Modelos SQLAlchemy, interfaces (ABCs), DTOs, value objects	Sin dependencias externas
`utils/`	Helpers, formatters, validadores reutilizables del dominio	Puede ser usado por cualquier capa
`container.py`	Instanciación y cableado de dependencias (DI manual)	Instancia todas las clases del dominio

---

MODELO DE BASE DE DATOS (MySQL en AWS RDS)

La base de datos es MySQL. Usar SQLAlchemy 2.x con PyMySQL como driver. Los modelos deben reflejar exactamente el siguiente esquema:

```
-- Usuarios del sistema
CREATE TABLE usuarios (
    id_usuario   INT AUTO_INCREMENT PRIMARY KEY,
    nombre       VARCHAR(100) NOT NULL,
    apellidos    VARCHAR(150),
    email        VARCHAR(150) UNIQUE,
    telefono     VARCHAR(50),
    direccion    VARCHAR(255),
    rol          VARCHAR(50)   -- valores: 'cliente', 'repartidor', 'gestor'
);

-- Clientes (extensión de usuarios)
CREATE TABLE clientes (
    id_cliente  INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario  INT UNIQUE,
    direccion   VARCHAR(255),
    telefono    VARCHAR(50),
    email       VARCHAR(150),
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
);

-- Bodegas
CREATE TABLE bodega (
    id_bodega      INT AUTO_INCREMENT PRIMARY KEY,
    nombre_bodega  VARCHAR(150) NOT NULL,
    ubicacion_gps  VARCHAR(100),
    direccion      VARCHAR(255)
);

-- Conductores / Repartidores
CREATE TABLE conductor (
    id_conductor  INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nombre        VARCHAR(100) NOT NULL,
    apellido      VARCHAR(150),
    nro_licencia  VARCHAR(100) UNIQUE,
    estado        VARCHAR(50)  -- valores: 'disponible', 'en_ruta', 'inactivo'
);

-- Vehículos
CREATE TABLE vehiculos (
    id_vehiculo   INT AUTO_INCREMENT PRIMARY KEY,
    conductor_id  INT NOT NULL,
    placa         VARCHAR(20) UNIQUE NOT NULL,
    modelo        VARCHAR(100),
    capacidad_kg  DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (conductor_id) REFERENCES conductor(id_conductor) ON DELETE SET
NULL
);

-- Pedidos / Paquetes
CREATE TABLE pedidos (
    id_pedido      INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario     INT,
    id_conductor   INT,
    id_bodega      INT,
    fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP,
    estado_actual  VARCHAR(50),  -- 'en_bodega','en_ruta','entregado','novedad','cancelado'
    FOREIGN KEY (id_usuario)   REFERENCES usuarios(id_usuario),
    FOREIGN KEY (id_conductor) REFERENCES conductor(id_conductor),
    FOREIGN KEY (id_bodega)    REFERENCES bodega(id_bodega)
);

-- Ítems de un pedido
CREATE TABLE item (
    id_item      INT AUTO_INCREMENT PRIMARY KEY,
    id_pedido    INT NOT NULL,
    descripcion  TEXT NOT NULL,
    peso         DECIMAL(10,2),
    cantidad     INT NOT NULL CHECK (cantidad > 0),
    FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido) ON DELETE CASCADE
);

-- Geolocalización GPS
CREATE TABLE geolocalizacion (
    id_geolocalizacion  INT AUTO_INCREMENT PRIMARY KEY,
    id_pedido           INT NOT NULL,
    latitud             DECIMAL(10,7) NOT NULL,
    longitud            DECIMAL(10,7) NOT NULL,
    descripcion_zona    VARCHAR(200),
    FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido) ON DELETE CASCADE
```

```
);

-- Seguimiento / historial de estados
CREATE TABLE seguimiento (
    id_seguimiento     INT AUTO_INCREMENT PRIMARY KEY,
    id_pedido          INT NOT NULL,
    id_geolocalizacion INT NOT NULL,
    fecha_hora         DATETIME DEFAULT CURRENT_TIMESTAMP,
    comentario         TEXT,
    FOREIGN KEY (id_pedido)         REFERENCES pedidos(id_pedido) ON DELETE
CASCADE,
    FOREIGN KEY (id_geolocalizacion) REFERENCES geolocalizacion(id_geolocalizacion)
ON DELETE CASCADE
);
```

ENDPOINTS A IMPLEMENTAR

Todos los endpoints deben tener el prefijo `/api/v1`. El API Gateway los orquesta externamente.

Dominio: Packages (Registro de Paquetes)

Método	Ruta	Descripción
`POST`	`/api/v1/packages`	Registra un nuevo paquete/pedido. Publica evento en SNS.
`GET`	`/api/v1/packages/{id_pedido}`	Obtiene detalle completo de un paquete.
`GET`	`/api/v1/packages`	Lista paquetes con filtros opcionales (`estado`, `id_conductor`).
`PATCH`	`/api/v1/packages/{id_pedido}/status`	Actualiza el estado del paquete. Publica evento en SNS.

Dominio: Tracking (Seguimiento)

Método	Ruta	Descripción
`GET`	`/api/v1/tracking/{id_pedido}`	Retorna el historial completo de seguimiento del paquete.
`GET`	`/api/v1/tracking/{id_pedido}/current-location`	Retorna la última ubicación GPS registrada.

POST	/api/v1/tracking/{id_pedido}/location	Registra una nueva coordenada GPS (llamado desde app repartidor).

Dominio: ETA (Tiempo Estimado de Entrega)

Método	Ruta	Descripción
GET	/api/v1/eta/{id_pedido}	Calcula y retorna el ETA del paquete.

Dominio: Notifications (Notificaciones)

Método	Ruta	Descripción
POST	/api/v1/notifications/subscribe	Suscribe un endpoint (email/sms/http) a un topic SNS.
POST	/api/v1/notifications/publish	Publica un mensaje manualmente a un topic SNS.
POST	/api/v1/notifications/webhook	Recibe confirmaciones de suscripción y mensajes de SNS (webhook AWS).

PRINCIPIOS DE DISEÑO A APLICAR

1. Interfaces como contratos entre capas

Cada dominio debe definir en `domain/interfaces.py` las interfaces (clases abstractas ABC) que la capa `infrastructure/` debe implementar:

```
# Ejemplo: packages/domain/interfaces.py
from abc import ABC, abstractmethod
from typing import Optional, List
from packages.domain.models import Pedido

class IPedidoRepository(ABC):
    @abstractmethod
    def find_by_id(self, id_pedido: int) -> Optional[Pedido]: ...

    @abstractmethod
    def find_all(self, filters: dict) -> List[Pedido]: ...

    @abstractmethod
    def save(self, pedido: Pedido) -> Pedido: ...
```

```
    @abstractmethod
    def update_status(self, id_pedido: int, estado: str) -> Optional[Pedido]: ...

class ISNSPublisher(ABC):
    @abstractmethod
    def publish(self, topic_arn: str, message: dict, subject: str) -> bool: ...
```

2. Inyección de dependencias en `container.py`

Cada dominio tiene un `container.py` que instancia y cablea todas las clases:

```
# Ejemplo: packages/container.py
from shared.database import get_db_session
from packages.infrastructure.repositories import PedidoRepository
from packages.infrastructure.sns_publisher import SNSPublisher
from packages.services.package_service import PackageService
from packages.presentation.controllers import PackageController

class PackageContainer:
    def __init__(self):
        self._db_session = get_db_session()
        self._pedido_repo = PedidoRepository(self._db_session)
        self._sns_publisher = SNSPublisher()
        self._package_service = PackageService(
            pedido_repository=self._pedido_repo,
            sns_publisher=self._sns_publisher
        )
        self._controller = PackageController(self._package_service)

    @property
    def controller(self) -> PackageController:
        return self._controller

# Singleton del contenedor
package_container = PackageContainer()
```

3. Segregación de interfaces (ISP)

No crear interfaces "todo en uno". Separar por responsabilidad:

- `IReadRepository` — solo operaciones de lectura
- `IWriteRepository` — solo operaciones de escritura
- `IEventPublisher` — publicación de eventos
- `INotificationSubscriber` — gestión de suscripciones

4. Principio de única responsabilidad (SRP)

- Un servicio = una responsabilidad. `PackageService` no gestiona notificaciones.
- Un repositorio = una entidad. `PedidoRepository` solo accede a la tabla `pedidos`.
- Un controlador = un dominio. No mezclar lógica de múltiples dominios en un endpoint.

5. Clean Code

- Nombres descriptivos en inglés para código, español solo en comentarios de negocio.
- Funciones con máximo 20 líneas.
- Type hints en todos los métodos y funciones.
- Docstrings en clases y métodos públicos.
- Sin números o strings mágicos: usar constantes o enums.

CONFIGURACIÓN Y VARIABLES DE ENTORNO

El archivo `shared/config.py` debe cargar todas las variables de entorno usando `python-dotenv`:

```
# Variables requeridas en .env
DB_HOST=rds-endpoint.us-east-1.rds.amazonaws.com
DB_PORT=3306
DB_NAME=ultima_milla
DB_USER=admin
DB_PASSWORD=secret

AWS_REGION=us-east-1
AWS_ACCESS_KEY_ID=AKIA...
AWS_SECRET_ACCESS_KEY=...
SNS_TOPIC_PACKAGE_EVENTS=arn:aws:sns:us-east-1:123456789:package-events
SNS_TOPIC_NOTIFICATIONS=arn:aws:sns:us-east-1:123456789:notifications
SQS_QUEUE_PACKAGES=https://sqs.us-east-1.amazonaws.com/123456789/packages-queue

FLASK_ENV=production
FLASK_PORT=5000
LOG_LEVEL=INFO
```

La cadena de conexión SQLAlchemy debe construirse así:
`mysql+pymysql://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}`

MANEJO DE ERRORES Y RESPUESTAS HTTP

Crear en `shared/http_responses.py` una clase helper para respuestas estandarizadas:

```
# Estructura estándar de respuesta exitosa
{
   "success": true,
   "data": { ... },
   "message": "Operación exitosa"
}

# Estructura estándar de respuesta de error
{
   "success": false,
   "error": {
      "code": "PACKAGE_NOT_FOUND",
      "message": "El paquete con id 123 no fue encontrado"
   }
}
```

Códigos HTTP a usar obligatoriamente:

Situación	Código
Creación exitosa	`201 Created`
Consulta exitosa	`200 OK`
Recurso no encontrado	`404 Not Found`
Error de validación / request inválido	`400 Bad Request`
Error de autenticación/autorización	`401 / 403`
Error interno del servidor	`500 Internal Server Error`
Servicio externo no disponible (AWS)	`503 Service Unavailable`

Crear excepciones personalizadas en `shared/exceptions.py`:

- `ResourceNotFoundException(id, resource_name)`
- `ValidationException(field, message)`

- `ExternalServiceException(service_name, detail)`
- `DatabaseException(operation, detail)`

LÓGICA DE NEGOCIO POR DOMINIO

Dominio Packages

`PackageService`:

- `register_package(data: dict) -> Pedido`: Crea el pedido en BD, publica evento `PaqueteRegistrado` en SNS topic de eventos.
- `get_package(id_pedido: int) -> Pedido`: Lee de BD con sus ítems asociados.
- `list_packages(filters: dict) -> List[Pedido]`: Lista con filtros opcionales.
- `update_status(id_pedido: int, new_status: str) -> Pedido`: Actualiza estado, publica evento `EstadoActualizado` en SNS.

Estados válidos del paquete (Enum): `EN_BODEGA` → `EN_RUTA` → `ENTREGADO` | `NOVEDAD` | `CANCELADO`

Dominio Tracking

`TrackingService`:

- `get_tracking_history(id_pedido: int) -> List[Seguimiento]`: Retorna todos los registros de seguimiento del paquete ordenados por fecha.
- `get_current_location(id_pedido: int) -> Geolocalizacion`: Retorna la última coordenada GPS registrada.
- `register_location(id_pedido: int, lat: float, lng: float, zona: str) -> Geolocalizacion`: Inserta nueva coordenada GPS y crea registro de seguimiento.

Dominio ETA

`ETAService`:

- `calculate_eta(id_pedido: int) -> dict`: Lógica básica de estimación:
  1. Verificar que el pedido existe y está `EN_RUTA`.
  2. Obtener la última ubicación GPS del repartidor.
  3. Retornar un ETA estimado con un delta fijo (implementación básica: retornar `estimated_minutes: 30` con la última ubicación y estado del pedido).
  4. Si el paquete ya fue `ENTREGADO`, retornar estado con fecha de entrega.

Dominio Notifications

`NotificationService`:

- `subscribe(endpoint: str, protocol: str, topic_arn: str) -> dict`: Suscribe al topic SNS usando boto3.
- `publish_event(topic_arn: str, message: dict, subject: str) -> bool`: Publica mensaje en SNS.
- `handle_webhook(payload: dict) -> bool`: Procesa confirmaciones de suscripción SNS y mensajes entrantes.

DOCKER Y DESPLIEGUE (AGNÓSTICO EKS/EC2)

Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "--timeout", "60", "wsgi:app"]
```

docker-compose.yml (para desarrollo local)

Incluir servicios:

- `api`: La aplicación Flask/Gunicorn
- Variables de entorno cargadas desde `.env`
- Healthcheck en `/api/v1/health`

Endpoint de salud obligatorio

GET /api/v1/health
Response 200: { "status": "healthy", "service": "ultima-milla-api", "version": "1.0.0" }

Preparación para Kubernetes (EKS)

- La aplicación no debe tener estado local (stateless).
- Configuración solo por variables de entorno (12-Factor App).
- El `Dockerfile` debe construir una imagen lista para subir a ECR (Amazon Container Registry).
- Crear un `k8s/` básico con `deployment.yaml` y `service.yaml` de ejemplo.

REQUIREMENTS.TXT

Incluir como mínimo:

```
flask>=3.0.0
flask-smorest>=0.44.0
SQLAlchemy>=2.0.0
PyMySQL>=1.1.0
boto3>=1.34.0
python-dotenv>=1.0.0
gunicorn>=21.2.0
marshmallow>=3.21.0
```

INSTRUCCIONES FINALES PARA CLAUDE CODE

1. **Crear toda la estructura de carpetas** descrita antes de escribir código.
2. **Comenzar por `shared/`**: config, database, exceptions, http_responses.
3. **Seguir el orden**: `domain/` → `infrastructure/` → `services/` → `presentation/` → `container.py`.
4. **Respetar las interfaces**: Nunca llamar directamente a una implementación desde una capa superior; siempre usar la interfaz ABC.
5. **Registrar todos los blueprints** en `app.py` usando `app.register_blueprint()`.
6. **No hardcodear** ningún valor de configuración. Todo desde variables de entorno.
7. **Crear el `.env.example`** con todas las variables necesarias sin valores reales.
8. **Crear un `README.md`** con instrucciones de instalación, configuración y ejecución local con Docker.
9. **Incluir comentarios** en secciones complejas explicando el propósito de negocio.
10. **Verificar** que la aplicación arranque correctamente ejecutando `python app.py` en modo desarrollo.